INSTITUTO SUPERIOR TECNOLÓGICO CORDILLERA

INFORME DE REGISTRO TRANSPARENTE Y JUSTIFICACIÓN

DEL USO DE IA GENERATIVA

Carrera: Desarrollo de Software

Asignatura: Programación Web 1

Estudiante: Josué Lucas

Fecha: 12 de Noviembre 2026

1. Declaración de Autoría y Comprensión Técnica

En cumplimiento con la **Política de Uso Ético y Responsable de Inteligencia Artificial** del Instituto Superior Tecnológico Cordillera, declaro que la herramienta de IA generativa fue utilizada como un asistente de aprendizaje, consulta conceptual y apoyo técnico para el diseño arquitectónico del proyecto.

Todo el código fuente resultante en HTML5, CSS3 y JavaScript fue revisado, adaptado, probado exhaustivamente en local y comprendido en su totalidad, garantizando la capacidad de sustentar cada bloque de código, función, estructura de datos y decisión de diseño en la evaluación técnica oral.

---

2. Bitácora de Registro Transparente de IA

Campo	Detalle
Herramienta utilizada	Asistente de IA (Antigravity / Gemini)
Fechas de interacción	08 al 12 de Noviembre de 2026
Objetivo académico	Asistencia en la estructuración de la lógica de persistencia sin base de datos (localStorage), integración de componentes interactivos (carrusel Bootstrap, SweetAlert2) y vinculación de roles (Cliente / Administrador).
Protección de Datos	Se utilizaron credenciales de prueba académicas (admin123, 123) y datos ficticios de vehículos y clientes. No se compartieron claves reales, tokens privados ni información institucional reservada.

Registro de Prompts Relevantes y Decisiones Técnicas

Prompt 1: Estructuración y Carrusel

- **Texto del Prompt:** *"En esta carpeta tengo que crear una pagina web de un taller de autos llamada DriveFix, soy estudiante y no debe ser profesional... haz un carrusel de imágenes... usa bootstrap para el carrusel y estilizar, que style.css sea aparte y java.js para interacción sencilla."*
- **Síntesis de la respuesta:** Se propuso la estructura base con Bootstrap 5, un carrusel de ancho completo (carousel slide), maquetación de servicios y un archivo JavaScript inicial para capturar eventos de click.
- **Qué se aprovechó:** La estructura del carrusel oficial con indicadores y controles previa/siguiente de Bootstrap, y la separación limpia en index.html, style.css y java.js.
- **Qué se descartó:** Se descartaron diseños iniciales sobrecargados o con estilos oscuros complejos para mantener una estética estudiantil, limpia y fácil de defender en clase.

Prompt 2: Sistema de Login, Roles y Persistencia sin Backend

- **Texto del Prompt:** *"Agrega un login con dos roles: admin (lista de clientes y CRUD de mantenimientos) y cliente (contratación de servicios, historial y estado de atención), ambos deben estar vinculados y comenzar con los registros en cero con opción a crear cuenta."*
- **Síntesis de la respuesta:** Creación de un modelo de persistencia del lado del cliente utilizando localStorage, funciones para almacenar y recuperar arrays de objetos serializados en formato JSON (getItem, setItem, JSON.parse, JSON.stringify), y lógica de enrutamiento basada en el rol de la sesión.
- **Qué se aprovechó:** La sincronización bidireccional: las solicitudes creadas por un cliente se almacenan en el array global de mantenimientos y son visualizadas y actualizadas por el administrador.
- **Qué se descartó:** Selectores manuales de rol en el formulario de login; el sistema ahora detecta el rol automáticamente al validar las credenciales.

Prompt 3: Cumplimiento de la Guía AD01 y Validación

- **Texto del Prompt:** *"Analiza la guía de la práctica y verifica que se cumpla todo: SweetAlert2, validaciones de placa, kilometraje, horarios, y campos del formulario."*
- **Síntesis de la respuesta:** Integración de la librería SweetAlert2 vía CDN para reemplazar los alert() y confirm() nativos, validación con expresiones regulares (Regex) para placas vehiculares, y validación de fechas (impidiendo fechas pasadas) y horarios de atención (08:00 a 18:00).
- **Qué se aprovechó:** Todas las alertas visuales amigables, las confirmaciones modales antes de eliminar registros en el CRUD y las validaciones de negocio exigidas en la rúbrica.

3. Justificación y Sustentación Técnica de lo Implementado

A. Lógica JavaScript (.js) y Persistencia con localStorage

Dado que en el nivel actual de la asignatura aún no se ha cubierto el desarrollo backend con bases de datos relacionales, la IA orientó el uso de la API web estándar **Web Storage (localStorage)**:

1. **Serialización y Deserialización de Objetos:** Para almacenar estructuras complejas (arrays de objetos de clientes y órdenes de trabajo), se utilizó JSON.stringify() para guardar strings en el navegador y JSON.parse() para recuperarlos como objetos manipulables.
2. **Métodos de Iteración y Búsqueda en Arrays (ES6):**
    - .find(): Utilizado en el inicio de sesión para buscar un cliente cuyas credenciales coincidan con las ingresadas.
    - .filter(): Utilizado en el portal del cliente para filtrar únicamente las órdenes cuyo clienteId pertenezca al usuario en sesión, garantizando que un cliente no vea datos de otros.

- .map(): Utilizado en el CRUD del Administrador para actualizar el estado o costo de un mantenimiento específico sin mutar de forma destructiva el resto del array.
- .forEach(): Empleado para recorrer los arrays e inyectar dinámicamente las filas de las tablas en el DOM mediante innerHTML.

3. **Control de Acceso Básico:** Se implementó verificación al inicio de admin.js y cliente.js leyendo localStorage.getItem("usuarioActualDriveFix"). Si un usuario intenta acceder a admin.html sin el rol adecuado, es redirigido automáticamente a login.html con una alerta de "Acceso Denegado".

B. Carrusel de Imágenes (Bootstrap 5)

Para el banner principal se aprovechó el componente nativo de **Bootstrap 5**:

- Clases estructurales: carousel slide, carousel-inner, carousel-item active, carousel-indicators y los botones carousel-control-prev / carousel-control-next.
- Atributo HTML5: data-bs-ride="carousel" para automatizar la transición entre diapositivas sin necesidad de librerías de terceros pesadas.

C. SweetAlert2 (Librería Externa)

Requerida explícitamente en la sección 5 de la guía técnica:

- Se implementó a través de CDN (sweetalert2@11).
- Permite lanzar diálogos asíncronos (Swal.fire({ ... }).then(...)) para confirmaciones antes de eliminar registros (CRUD - Delete) y mensajes de éxito con temporizador al iniciar sesión o registrar solicitudes.

D. Integración de Servicios Externos (APIs / Embebidos)

1. **Google Maps Embed:** Se integró un <iframe> responsivo con la ubicación geográfica del taller en la página de contacto para que el cliente pueda geolocalizar el establecimiento.
2. **WhatsApp Click to Chat API:** Se estructuró el protocolo estándar https://api.whatsapp.com/send?phone=...&text=... tanto en un botón visible como en un botón flotante, permitiendo al cliente iniciar una conversación directa con el taller mecánico.

4. Fuentes Bibliográficas Verificadas

1. **MDN Web Docs (Mozilla Developer Network):**

- *Window.localStorage*: Documentación oficial sobre almacenamiento web del lado del cliente.
  URL: https://developer.mozilla.org/es/docs/Web/API/Window/localStorage
- *Array.prototype (filter, find, map, forEach)*: Métodos de manipulación de arrays en JavaScript moderno.
  URL:
  https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Array

2. **Bootstrap 5 Official Documentation:**
   - *Carousel Component*: Especificación oficial de clases y atributos data para carruseles responsivos.
     URL: https://getbootstrap.com/docs/5.3/components/carousel/
3. **SweetAlert2 Documentation:**
   - *SweetAlert2 Guide*: Guía de uso, opciones de configuración, promesas y personalización de modales.
     URL: https://sweetalert2.github.io/
4. **WhatsApp Brand Resource & Click to Chat API:**
   - *Cómo usar el enlace directo de WhatsApp*: Especificación para crear enlaces con mensaje predeterminado.
     URL: https://faq.whatsapp.com/5913398998672934